

□ Лекция 1. Android Build Journey: как код превращается в приложение

□ Цель лекции

Понять полный путь Android-приложения: от Kotlin-кода до запуска на устройстве. Разобраться, что реально происходит при нажатии кнопки **Run** в Android Studio.

□ Главная идея

Для новичка кнопка **Run** выглядит как магия:

нажал → приложение открылось

Но на самом деле это цепочка этапов сборки, компиляции, упаковки и запуска.

□ Общая схема процесса

```
□ Kotlin (.kt)
  ↓
Kotlin compiler (kotlinc)
  ↓
Java bytecode (.class)
  ↓
D8 compiler
  ↓
DEX files (Dalvik bytecode)
  ↓
aapt2 + Gradle packaging
  ↓
APK / AAB
  ↓
Signing (keystore)
  ↓
ADB install
```

↓

Android Runtime (ART) запускает приложение

1. Компиляция кода (Kotlin → bytecode → DEX)

1.1 Kotlin компилятор (kotlinc)

Первый шаг:

- `.kt` файлы превращаются в Java bytecode (`.class`)

Важно понимать:

Kotlin сам по себе не выполняется на Android напрямую.

1.2 D8: превращение в DEX

Дальше вступает D8 (или старый dx).

Он делает ключевую вещь:

превращает bytecode в формат Android - DEX

Что такое DEX

DEX = Dalvik Executable

Это формат, оптимизированный под Android Runtime (ART).

Практический смысл

- один проект → много `.class`
- они сжимаются в `classes.dex`

Типичная проблема

✗ "Too many methods (64K limit)"

Причина:

- слишком много методов в DEX

Решение:

- MultiDex

2. Сборка приложения (Gradle + aapt2)

2.1 Gradle - координатор сборки

Gradle управляет всем процессом:

- компиляцией
- зависимостями
- ресурсами
- сборкой APK/AAB

2.2 aapt2 - упаковка ресурсов

Он делает важную работу:

- компилирует XML → бинарный формат
- обрабатывает ресурсы (*res/*)
- генерирует класс *R*

Пример ресурсов

```
□ res/  
  ├── layout/  
  ├── drawable/  
  └── values/
```

☒ Как мы обращаемся к ресурсам

```
□ setContentView(R.layout.activity_main)
  getString(R.string.app_name)
```

❗ Ошибка новичков

✗ Используют “магические строки” вместо ресурсов

3. Упаковка APK / AAB

3.1 Что такое APK

APK = готовый установочный файл Android-приложения

3.2 Что внутри APK

- classes.dex
- ресурсы (res)
- AndroidManifest.xml
- библиотеки (.so)

3.3 AAB (Android App Bundle)

Современный формат.

Разница:

APK	AAB
один файл	набор модулей
всё внутри	динамическая сборка

Важно

Google Play теперь требует AAB

4. Подпись приложения (Signing)

Почему это нужно

Android не установит приложение без подписи.

Keystore

Это файл, который содержит:

- приватный ключ
- сертификат разработчика

Критическая ошибка

✗ потерял keystore → потерял возможность обновлять приложение

Типы подписей

- debug (для разработки)
- release (для продакшена)

5. Установка через ADB

После сборки APK:

□ APK → ADB → устройство

☒ ADB (Android Debug Bridge)

Это инструмент для взаимодействия с устройством.

Что он делает

- устанавливает приложения
- удаляет приложения
- читает логи

- управляет устройством

Типичный пример

❑ `adb install app.apk`

6. Запуск приложения внутри Android

6.1 Zygote - стартовая система

Android использует специальный процесс:

Zygote = “шаблон процессов”

Почему он важен

Он уже содержит:

- базовые классы Android
- системные библиотеки

Механика

❑ Zygote → fork → новый процесс приложения

6.2 ActivityManagerService (AMS)

AMS делает:

- создание процесса приложения
- запуск Application
- запуск Activity

Итог запуска

❑ Application → Activity → UI экран

❑

7. Где чаще всего ломается сборка

7.1 Проблемы компиляции

- конфликт зависимостей
- несовместимые версии Kotlin

7.2 Проблемы ресурсов

- дублирующиеся имена
- битые XML

7.3 Проблемы DEX

- 64K методов
- слишком много библиотек

7.4 Проблемы подписи

- неверный keystore
- потерял пароль

8. Практическое понимание (очень важно)

Что должен понимать разработчик

Когда ты нажимаешь Run:

Android Studio не “запускает код”
она строит полноценный установочный продукт

Простая аналогия

Этап	Аналогия
------	----------

Kotlin → DEX перевод книги на другой язык

Gradle build издательство

APK готовая книга

ADB install доставка книги

Zygote печатный станок

9. Советы (как у senior разработчиков)

Совет 1

Если сборка медленная:

- смотри Gradle cache
- проверяй зависимости

Совет 2

Если приложение не ставится:

- проверяй подпись
- проверяй minSdk

Совет 3

Если крашится сразу:

- смотри Logcat
- проверяй Application.onCreate()

10. Итог лекции

Главное понимание

□ Android build = pipeline превращения кода в установленное приложение

☒ Что важно запомнить

- Kotlin не выполняется напрямую
- DEX - ключевой формат Android
- Gradle управляет всем процессом
- APK/AAB - результат сборки
- Zygote отвечает за запуск процессов

Если упростить

Android-разработка начинается не с UI, а с понимания того, как код становится приложением

□ Лекция 2. Структура Android-проекта и Gradle

□ Цель лекции

Понять, из чего состоит Android-проект, как Gradle управляет сборкой и почему половина “странных ошибок” у новичков на самом деле связана именно со структурой проекта и конфигурацией сборки.

□ Главная идея

Android-проект - это не просто папки с кодом.

Это **система модулей + ресурсов + конфигурации сборки**, где Gradle выступает как “диспетчер производства”.

1. Общая структура Android-проекта

□ Базовая структура

□ MyApp/
|— app/
|— gradle/
|— build.gradle (project)
|— settings.gradle
|— gradle.properties

□ Основная работа почти всегда происходит в модуле:

□ app/
□

2. Модуль app - сердце приложения

□ Внутри app/

□ app/

└── manifests/

└── java/ или kotlin/

└── res/

└── assets/

└── build.gradle

□

2.1 manifests / AndroidManifest.xml

Это “паспорт приложения”.

Он отвечает за:

- какие экраны существуют
 - какие разрешения нужны
 - какая Activity стартует первой
-

Пример

□ <application>

```
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

</application>
```

□

❑ Частая ошибка новичков

✗ добавили Activity в код, но забыли зарегистрировать в Manifest

→ экран просто “не открывается”

3. res/ - ресурсы приложения

❑ Структура ресурсов

❑ res/

├── layout/

├── drawable/

├── values/

├── mipmap/

└── menu/

❑

3.1 layout

XML-разметка экранов.

❑ activity_main.xml

❑

3.2 values

Хранит “данные без логики”:

- strings.xml
- colors.xml
- themes.xml

Почему это важно

Если ты хардкодишь строки в коде:

- ✗ нельзя перевести приложение
- ✗ сложно поддерживать

3.3 drawable

- иконки
- картинки
- shape-объекты (градиенты, рамки)

❑ Ошибка новичков

✗ кладут изображения в код или неправильную папку
→ ресурсы не находятся → crash или blank UI

4. Класс R - мост между кодом и ресурсами

Что делает R

Gradle автоматически генерирует:

```
❑ R.layout.activity_main  
R.string.app_name  
R.drawable.icon
```



Важно

R создаётся автоматически при сборке.

❑ Частая проблема

✗ "Unresolved reference R"

Причины:

- ошибка в XML
 - не собрался проект
 - конфликт ресурсов
-

5. Gradle - сердце Android-сборки

❑ Что такое Gradle

Gradle - это система автоматической сборки.

Он управляет:

- зависимостями
 - версиями SDK
 - сборкой APK/AAB
 - build types
-

6. Уровни Gradle-файлов

6.1 project-level build.gradle

Глобальные настройки проекта

```
repositories
plugins
common configs

```

6.2 module-level build.gradle (app)

Главный файл для Android-разработчика.

7. Ключевые настройки Android Gradle

7.1 compileSdk / minSdk / targetSdk

Параметр	Значение
compileSdk	с чем собираем
minSdk	минимальная версия
targetSdk	оптимизация под версию

□ Реальный кейс

- minSdk слишком высокий → часть пользователей отваливается
 - targetSdk устарел → Play Store ругается
-

8. dependencies - внешние библиотеки

Пример

```
dependencies {  
    implementation("androidx.core:core-ktx")  
    implementation("androidx.recyclerview:recyclerview")  
}
```

Что происходит на самом деле

Gradle:

- скачивает библиотеки
 - подключает их в проект
 - добавляет в DEX при сборке
-

□ Типичная проблема

✗ конфликт версий библиотек

→ "Duplicate class found"

9. Build Types

Основные типы

Тип	Назначение
-----	------------

debug разработка

release продакшен

Пример

```
□ buildTypes {  
    release {  
        isMinifyEnabled = true  
    }  
}
```

Что важно

- debug = быстро, но “тяжело”
 - release = оптимизировано
-

10. Product Flavors (реальные проекты)

Используются для разных версий приложения:

- dev
 - stage
 - prod
-

Пример

□ app-dev
app-prod
app-stage



Реальный кейс

Один код → разные серверы:

- dev.api.com
 - prod.api.com
-

11. Gradle.properties

Используется для:

- оптимизации сборки
 - настройки памяти Gradle
-

Пример

```
org.gradle.jvmargs=-Xmx4g  
android.useAndroidX=true
```

12. Как реально работает сборка проекта

Полный pipeline

```
Gradle  
→ dependency resolve
```

- compile Kotlin
- process resources (aapt2)
- merge manifests
- DEX conversion
- packaging APK/AAB
- signing

□

13. Частые проблемы в проектах

13.1 Gradle sync failed

Причины:

- интернет
 - сломанные зависимости
 - конфликт версий
-

13.2 Manifest merge conflict

Когда:

- 2 библиотеки добавляют одинаковые настройки
-

13.3 Resource not found

- неправильное имя
 - не собрался R
-

13.4 Duplicate class error

- конфликт библиотек
-

14. Как думает senior разработчик

При любой ошибке он проверяет:

1. Gradle sync
 2. dependencies
 3. resources
 4. manifest
 5. build cache
-

Важное понимание

80% Android ошибок - это не код, а конфигурация проекта

15. Практическая модель мышления

Как нужно смотреть на проект

☐ Project = (Code + Resources + Dependencies + Build System)
☐

Если ломается приложение:

- код? редко
- Gradle? часто
- ресурсы? очень часто

16. Итог лекции

Главное понимание

- Android проект = структура + ресурсы + Gradle конфигурация
-

Что нужно запомнить

- Manifest управляет приложением
- res/ - это UI и данные
- R генерируется автоматически
- Gradle управляет всей сборкой
- dependencies = внешняя система проекта

Если упростить

Android-разработка - это не только код, это управление системой сборки и ресурсами

□ Лекция 3. RecyclerView и списки в Android

□ Цель лекции

Понять, как Android отображает списки данных, почему старый `ListView` больше не используют, и как правильно работать с `RecyclerView`, чтобы приложение не лагало на 1000+ элементов.

□ Главная идея

Списки - это основа Android-приложений:

- чаты (Telegram, WhatsApp)
- ленты (Instagram, TikTok)

- каталоги товаров (маркетплейсы)
- уведомления

И почти всегда ты работаешь не с “одним экраном”, а с **повторяющимися элементами**.

1. Проблема списков в Android

✗ Наивный подход (ListView)

Раньше использовали `ListView`.

Проблема:

- каждый элемент создавался заново
 - при скролле всё пересоздавалось
 - лаги на длинных списках
-

□ Что происходило внутри

□ `scroll → create view → bind data → destroy → repeat`

□

Главная проблема

система постоянно создаёт и уничтожает View

Итог

- плохая производительность
- дерганный скролл
- перерасход памяти

2. Появление RecyclerView

□ Идея RecyclerView

Название говорит само за себя:

Recycler = “перерабатывать View”

Основной принцип

□ создал View → переиспользовал → перерисовал данные

□

Что это даёт

- меньше объектов
 - плавный скролл
 - стабильная память
-

3. Архитектура RecyclerView

RecyclerView состоит из 3 ключевых частей:

□ RecyclerView

↓

Adapter

↓

ViewHolder

↓



4. Adapter - мост между данными и UI

Что делает Adapter

Adapter отвечает за:

- создание View
 - привязку данных
 - управление списком
-

Логика простыми словами

Adapter = “переводчик между списком и экраном”

Основные методы

☐ onCreateViewHolder()
onBindViewHolder()
getItemCount()
☐

Что происходит

Метод	Когда вызывается
onCreateViewHolder	создаём View
onBindViewHolder	заполняем данными

getItemCount

размер списка

5. ViewHolder - ключ к производительности

Проблема без ViewHolder

✗каждый раз искать View через `findViewById`

Это дорого.

Решение

ViewHolder хранит ссылки на UI элементы.

Принцип

□ создал View → сохранил ссылки → переиспользовал
□

Пример

```
□ class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {  
    val name = view.findViewById<TextView>(R.id.name)  
}  
□
```

Почему это важно

- меньше операций поиска
 - быстрее скролл
 - меньше нагрузка на CPU
-

6. LayoutManager - как рисуются элементы

Что он делает

LayoutManager решает:

- как расположить элементы
 - в каком направлении
 - как прокручивать
-

Основные типы

Тип	Описание
LinearLayoutManager	список
GridLayoutManager	сетка
StaggeredGridLayoutManager	“плитка”

Пример

Instagram = GridLayoutManager

Telegram = LinearLayoutManager

7. Полный пример RecyclerView

Adapter

```
class UserAdapter(private val users: List<String>) :  
    RecyclerView.Adapter<UserAdapter.VH>() {  
  
    class VH(view: View) : RecyclerView.ViewHolder(view) {  
        val name: TextView = view.findViewById(R.id.name)  
    }  
  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):  
VH {  
        val view = LayoutInflater.from(parent.context)  
            .inflate(R.layout.item_user, parent, false)  
        return VH(view)  
    }  
  
    override fun onBindViewHolder(holder: VH, position: Int) {  
        holder.name.text = users[position]  
    }  
  
    override fun getItemCount() = users.size  
}
```

8. Как RecyclerView работает внутри

Внутренний цикл

```
scroll  
→ reuse ViewHolder  
→ bind new data
```

→ display



Главное отличие от ListView

ListView	RecyclerView
пересоздаёт View	переиспользует
медленный	быстрый
устаревший	стандарт

9. DiffUtil и ListAdapter (уровень middle)

Проблема

Если ты обновляешь список:

✗ RecyclerView перерисует всё

Решение - DiffUtil

Он сравнивает:

- старый список
- новый список

и обновляет только изменения

Пример логики

```
□ old list → compare → new list → update only changed items  
□
```

ListAdapter

Это готовая реализация DiffUtil.

10. Частые ошибки новичков

✗1. пересоздание адаптера

Создают новый Adapter при каждом обновлении

→ ломается производительность

✗2. тяжёлая логика в onBindViewHolder

Там должно быть только:

- присвоение данных
-

✗3. отсутствие ViewHolder

→ лаги при скролле

✗4. использование notifyDataSetChanged()

→ перерисовывает весь список

11. Реальный кейс из практики

Проблема

- список из 2000 товаров
 - скролл дергается
-

Причина

- нет ViewHolder
 - используется notifyDataSetChanged()
-

Решение

- ViewHolder
 - DiffUtil
 - оптимизация layout
-

Итог

- скролл стал плавным
 - CPU нагрузка упала
-

12. Когда RecyclerView “плохой”

RecyclerView может лагать если:

- тяжёлый XML layout
 - картинки без кеша
 - сложная логика биндинга
-

13. Как думает senior разработчик

При работе со списками он думает:

- можно ли переиспользовать View?
 - что происходит при скролле?
 - где создаются объекты?
 - можно ли уменьшить bind logic?
-

14. Итог лекции

Главное понимание

☐ RecyclerView = система переиспользования View + адаптация данных + оптимизация скrolла

☐

Что нужно запомнить

- ListView устарел
- RecyclerView = стандарт
- ViewHolder = производительность

- Adapter = мост данных и UI
 - LayoutManager = структура отображения
-

Если упростить

RecyclerView существует, чтобы Android не пересоздавал UI при каждом скролле

⚡ Лекция 4. Coroutines и асинхронность в Android

□ Цель лекции

Понять, почему Android “виснет”, что такое Main Thread, и как корутины позволяют писать асинхронный код без колбэков, лагов и хаоса.

□ Главная идея

Android-приложение работает по очень строгому правилу:

! UI должен обновляться только в Main Thread

! Любая тяжёлая операция на Main Thread = лаг или ANR

1. Что такое асинхронность

✗ Проблема синхронного кода

□ UI thread → network request → wait → freeze UI → ANR

□

□ Что происходит

Пока идёт запрос:

- кнопки не нажимаются
 - экран не реагирует
 - система думает, что приложение “умерло”
-

□ ANR (Application Not Responding)

Если UI не отвечает ~5 секунд → Android показывает:

“Приложение не отвечает. Закрыть?”

2. Main Thread - главный враг новичка

Что работает в Main Thread:

- UI (TextView, RecyclerView)
 - клики
 - анимации
-

Что нельзя делать:

✗ сеть

✗ база данных

✗ тяжёлые вычисления

✗ парсинг больших JSON

Простая модель

□ Main Thread = "пульт управления UI"

□

3. Как было раньше (до корутин)

Callback hell

```
□ api.getUser(object : Callback {  
    override fun onSuccess(user: User) {  
        api.getPosts(user.id, object : Callback {  
            override fun onSuccess(posts: List<Post>) {  
                api.getComments(posts[0].id, object : Callback {  
                    })  
                }  
            })  
        }  
    })  
})
```



❑ Проблема

- нечитаемо
 - сложно дебажить
 - легко ошибиться
-

4. Появление Coroutines

❑ Что такое корутины

Корутины - это способ писать асинхронный код:

как синхронный, но без блокировки потока

Главное отличие

Threads	Coroutines
тяжёлые	лёгкие
дорогие	дешёвые
OS управляет	Kotlin управляет

5. Простая корутина

```
❑viewModelScope.launch {
```

```
    val user = api.getUser()  
    show(user)  
}  
☐
```

Что происходит

☐ launch → suspend function → resume → update UI
☐

6. suspend functions

Что значит suspend

☐ suspend fun getUser(): User
☐

Это НЕ значит:

✗ что функция "в фоне"

Это значит:

функция может "приостановиться" и продолжиться позже

Важно

suspend ≠ thread

7. Dispatchers - где выполняется код

Основные потоки

Dispatcher	Где используется
Main	UI
IO	сеть, БД
Default	CPU задачи

Пример

```
□ withContext(Dispatchers.IO) {  
    api.getUsers()  
}  
□
```

□ Ошибка новичков

✗ делают сеть в Main Thread

→ лаги

8. viewModelScope и lifecycleScope

Почему это важно

Корутины должны умирать вместе с экраном.

viewModelScope

Живёт пока жив ViewModel

```
□ viewModelScope.launch {  
}  
□
```

lifecycleScope

Живёт пока жив Activity/Fragment

□ Проблема без scope

- утечки памяти
 - продолжающиеся запросы после закрытия экрана
-

9. structured concurrency

□ Главная идея

все корутины должны иметь родителя

Пример

```
□ viewModelScope.launch {  
    launch { task1() }  
    launch { task2() }  
}
```

□

Если родитель умер:

→ все дочерние корутины отменяются

10. async / await

Когда нужно

Когда нужно параллельно выполнить задачи

```
□ val user = async { api.getUser() }  
val posts = async { api.getPosts() }  
  
val result = user.await() + posts.await()  
□
```

□ Ошибка новичков

✗ Используют async без необходимости

→ усложняют код

11. Cancellation (очень важно)

Корутины можно отменить

```
□ job.cancel()
```

```
□
```

Когда это происходит автоматически

- пользователь ушёл со страницы
 - ViewModel уничтожена
 - lifecycle изменился
-

Почему это важно

Без cancellation:

- утечки сети
 - лишние запросы
 - перегрузка API
-

12. Flow - реактивные потоки данных

□ Что это

Flow = поток значений во времени

Пример

```
□ flow {  
    emit(1)  
    emit(2)  
    emit(3)  
}
```




Где используется

- UI state
 - загрузка данных
 - события
-

13. StateFlow vs SharedFlow

StateFlow

хранит текущее состояние

Пример:

- экран загрузки
 - список пользователей
-

SharedFlow

события

Пример:

- показать Toast
 - навигация
 - ошибки
-

14. Типичный MVVM с корутинами

```
viewModelScope.launch {  
    _state.value = Loading  
  
    try {  
        val data = withContext(Dispatchers.IO) {  
            api.getUsers()  
        }  
        _state.value = Success(data)  
    } catch (e: Exception) {  
        _state.value = Error(e.message)  
    }  
}  
□
```

15. Частые ошибки новичков

✗1. блокировка Main Thread

→ ANR

✗2. забыли withContext(IO)

→ сеть на UI

✗3. утечки корутин

→ нет scope

✗4. async без await

→ потеря результата

16. Реальный кейс из практики

Проблема

- приложение зависает при открытии экрана
-

Причина

```
❑ api.getUsers() // выполняется в Main Thread  
❑
```

Решение

```
❑ withContext(Dispatchers.IO) {  
    api.getUsers()  
}  
❑
```

Итог

- UI стал плавным
 - исчез ANR
 - уменьшилась нагрузка
-

17. Как думает senior разработчик

При любой задаче он спрашивает:

- где выполняется код?
 - можно ли это отменить?
 - не блокируется ли UI?
 - какой score у корутины?
-

18. Итог лекции

Главное понимание

□ Coroutines = способ выполнять тяжёлую работу без блокировки UI и без потокового хаоса

□

Что нужно запомнить

- Main Thread нельзя блокировать
 - suspend ≠ background thread
 - Dispatchers управляют контекстом
 - viewModelScope = безопасный запуск
 - Flow = реактивные данные
-

Если упростить

Корутины - это способ сделать Android быстрым и не “зависшим”

□ Лекция 5. Retrofit и работа с сетью (с корутинами)

□ Цель лекции

Понять, как Android-приложение общается с сервером, как устроен REST API, и как Retrofit вместе с корутинами превращает сетевые запросы в простой и безопасный код без callback-хаоса.

□ Главная идея

Любое современное приложение - это:

- клиент, который постоянно общается с сервером

Чат, лента, авторизация, покупки - всё через сеть.

1. Как выглядит типичная схема

□ Android App → HTTP Request → Backend API → Database → Response → Android App
□

Что важно понимать QA/разработчику

- приложение НЕ хранит “истину”
 - сервер всегда главный источник данных
 - сеть = нестабильная среда
-

2. REST API простыми словами

REST - это способ общения через HTTP.

Основные методы

Метод	Что делает
GET	получить данные
POST	создать
PUT	заменить полностью
PATCH	изменить частично

DELETE удалить

Пример

□ GET /users/1

□

Ответ:

```
□ {  
  "id": 1,  
  "name": "Ivan"  
}
```

□

3. Что такое Retrofit

□ Простое объяснение

Retrofit - это:

библиотека, которая превращает HTTP-запросы в Kotlin-интерфейсы

Без Retrofit:

✗ ручной HTTP

✗ обработка потоков

✗ парсинг JSON вручную

C Retrofit:

- ✓ один интерфейс
 - ✓ аннотации
 - ✓ автоматический JSON → object
-

4. Базовая архитектура Retrofit

□ ViewModel → Repository → Retrofit → API → Server
□

5. Модель данных (DTO)

Пример ответа сервера

```
□ {  
  "id": 1,  
  "name": "Ivan",  
  "email": "ivan@mail.com"  
}
```

□

Kotlin модель

```
□ data class User(  
    val id: Int,  
    val name: String,  
    val email: String  
)
```

□

❑ Ошибка новичков

- ✗ не учитывают null
 - ✗ не синхронизируют поля с API
-

6. API интерфейс Retrofit

❑ Главное понимание

Retrofit работает через интерфейсы.

```
❑ interface ApiService {  
  
    @GET("users")  
    suspend fun getUsers(): List<User>  
  
    @GET("users/{id}")  
    suspend fun getUserById(@Path("id") id: Int): User  
  
    @POST("users")  
    suspend fun createUser(@Body user: User): User  
}  
❑
```

Что здесь важно

- @GET, @POST = HTTP методы
 - @Path = вставка в URL
 - @Body = JSON в запросе
-

7. Retrofit + Coroutines

□ Главная сила

Retrofit поддерживает `suspend` функции.

Пример

```
□ suspend fun getUsers(): List<User>  
□
```

Что это даёт

- ✓нет callbacks
 - ✓нет threading ручного
 - ✓читаемый код
-

8. Создание Retrofit клиента

```
□ val retrofit = Retrofit.Builder()  
    .baseUrl("https://api.example.com/")  
    .addConverterFactory(GsonConverterFactory.create())  
    .build()
```

```
val api = retrofit.create(ApiService::class.java)
```

```
□
```

Что происходит внутри

- создаётся HTTP клиент
 - подключается JSON parser
 - генерируется реализация интерфейса
-

9. Использование в корутинах

ViewModel пример

```
class UserViewModel(  
    private val api: ApiService  
) : ViewModel() {  
  
    fun loadUsers() {  
        viewModelScope.launch {  
            try {  
                val users = withContext(Dispatchers.IO) {  
                    api.getUsers()  
                }  
                println(users)  
            } catch (e: Exception) {  
                println("Error: ${e.message}")  
            }  
        }  
    }  
}
```

10. Почему withContext(IO) важен

Без него

✗ сеть в Main Thread → ANR

С ним

✓ сеть в фоне

✓ UI остаётся отзывчивым

11. OkHttp - двигатель Retrofit

Retrofit сам ничего не отправляет.

Он использует:

OkHttp - HTTP-клиент низкого уровня

Возможности OkHttp

- таймауты
 - логирование
 - interceptors
 - кеширование
-

12. Interceptors (очень важно в реальных проектах)

Пример добавления токена

```
❑ val interceptor = Interceptor { chain ->
    val request = chain.request()
        .newBuilder()
        .addHeader("Authorization", "Bearer token")
        .build()

    chain.proceed(request)
}
❑
```

Где используется

- авторизация
 - логирование запросов
 - аналитика
-

13. Логирование запросов

```
❑ val logging = HttpLoggingInterceptor().apply {
    level = HttpLoggingInterceptor.Level.BODY
}
❑
```

Что видно

- URL
 - headers
 - body
 - response
-

14. Ошибки сети (очень важно для QA)

Типы ошибок

Код	Значение
400	ошибка запроса
401	нет авторизации
403	запрещено
404	не найдено
500	ошибка сервера

Реальный кейс

- 401 → токен протух
- 500 → backend сломался
- 404 → неправильный endpoint

15. Обработка ошибок в корутинах

```
try {  
    val users = api.getUsers()  
} catch (e: HttpException) {  
    println("Server error")  
} catch (e: IOException) {  
    println("No internet")  
}
```

□

16. Типичный production-паттерн

□ UI → ViewModel → Repository → API → Result wrapper

□

Пример Result

```
□ sealed class Result<out T> {  
    data class Success<T>(val data: T): Result<T>()  
    data class Error(val message: String): Result<Nothing>()  
}
```

□

17. Частые ошибки новичков

✗1. сеть в Main Thread

→ ANR

✗2. нет try/catch

→ crash при ошибке API

✗3. игнорирование HTTP кодов

→ приложение думает, что всё ОК

✗4. отсутствие timeout

→ бесконечное ожидание

18. Реальный кейс из практики

Проблема

- экран загрузки зависает
-

Причина

```
❑ api.getUsers() // без IO dispatcher  
❑
```

Исправление

```
❑ withContext(Dispatchers.IO) {  
    api.getUsers()  
}  
❑
```

Итог

- исчезли фриззы
- улучшилась отзывчивость UI

19. Как думает senior разработчик

Перед любым сетевым кодом он спрашивает:

- это suspend или callback?
 - где выполняется запрос?
 - что будет при отсутствии сети?
 - как обрабатывается ошибка?
 - можно ли это отменить?
-

20. Итог лекции

Главное понимание

☐ Retrofit = способ безопасно и удобно делать HTTP-запросы через Kotlin + корутины
☐

Что нужно запомнить

- Retrofit = HTTP abstraction
 - suspend = нативная асинхронность
 - IO dispatcher = обязателен для сети
 - OkHttp = транспортный слой
 - ошибки сети = нормальная часть системы
-

Если упростить

Retrofit + Coroutines = “сеть как обычный код, но без блокировки UI”

□ Лекция 6. Room и локальная база данных в Android

□ Цель лекции

Понять, как Android хранит данные оффлайн, зачем нужна локальная база данных и как Room решает проблемы SQLite так, чтобы с ней можно было работать без боли.

□ Главная идея

Сервер - не всегда доступен.

Поэтому любое нормальное приложение должно уметь:

- работать без интернета и не терять данные после перезапуска
-

1. Зачем нужна локальная база данных

Реальные сценарии

- чат должен показывать сообщения без сети
 - маркетплейс хранит корзину
 - приложение кеширует пользователей
 - оффлайн-режим
-

✗ Без локальной БД

- всё грузится каждый раз с сервера
 - нет оффлайн режима
 - медленный UI
-

✓ С локальной БД

- мгновенный доступ к данным
 - работа без интернета
 - меньше нагрузки на API
-

2. Что такое Room

☐ Простое объяснение

Room - это:

удобная надстройка над SQLite, которая убирает ручной SQL хаос

Без Room:

- ✗ ручной SQL
 - ✗ курсоры
 - ✗ ошибки в запросах
 - ✗ нет типобезопасности
-

C Room:

- ✓ Kotlin-объекты
 - ✓ проверка запросов на compile-time
 - ✓ меньше ошибок
-

3. Архитектура Room

Room состоит из 3 частей:

□ Entity → DAO → Database
□

4. Entity - таблица в виде класса

□ Что это

Entity = таблица базы данных

Пример

```

@Entity(tableName = "users")
data class UserEntity(
    @PrimaryKey val id: Int,
    val name: String,
    val email: String
)

```

Что происходит

SQL Room

table Entity

column field

❑ Ошибка новичков

✗ путают Entity и API model

5. DAO - доступ к данным

❑ Что это

DAO = слой запросов к базе

Пример

❑ @Dao

```
interface UserDao {  
  
    @Query("SELECT * FROM users")  
    fun getUsers(): Flow<List<UserEntity>>  
  
    @Insert(onConflict = OnConflictStrategy.REPLACE)  
    suspend fun insert(user: UserEntity)  
  
    @Delete  
    suspend fun delete(user: UserEntity)  
}  
□
```

Почему Flow

Flow даёт:

- автоматическое обновление UI
 - реактивность
 - live данные
-

6. Database - точка входа

```
□@Database(  
    entities = [UserEntity::class],  
    version = 1  
)  
abstract class AppDatabase : RoomDatabase() {  
  
    abstract fun userDao(): UserDao  
}  
□
```

Что делает Database

- создаёт SQLite внутри
 - управляет версиями
 - связывает DAO и Entity
-

7. Как создаётся база

```
❑ val db = Room.databaseBuilder(  
    context,  
    AppDatabase::class.java,  
    "app_db"  
).build()  
❑
```

8. Где хранится база

- ❑ Внутри устройства:
 - /data/data/your_app/
 - файл SQLite базы
-

Важно

- база приватная для приложения
 - другие приложения не имеют доступа
-

9. Поток данных в Room

❑ UI → ViewModel → Repository → DAO → SQLite

□

10. Пример реального сценария

Загрузка пользователей

□ `fun loadUsers() = userDao.getUsers()`

□

UI автоматически обновляется

Потому что Flow “слушает” изменения базы

11. Кеширование данных (очень важно)

Правильная схема

□ `API → Room DB → UI`

□

Логика

1. сначала показываем данные из Room
 2. потом обновляем с сервера
 3. сохраняем обратно в Room
-

□ Это стандарт production

12. Offline-first подход

□ Что это

Приложение работает сначала с локальной БД, потом синхронизируется с сервером

Пример

- Telegram показывает сообщения даже без интернета
 - потом догружает новые
-

13. Версионирование базы

Проблема

Ты изменил Entity:

□ `val age: Int`

□

Нужно

увеличить version

□ `@Database(version = 2)`

□

И добавить migration

14. Migration (очень важно)

```
❑ val MIGRATION_1_2 = object : Migration(1, 2) {  
    override fun migrate(db: SupportSQLiteDatabase) {  
        db.execSQL("ALTER TABLE users ADD COLUMN age INTEGER")  
    }  
}  
❑
```

❑ Ошибка новичков

✗ меняют Entity без миграции

→ приложение падает

15. Room vs SharedPreferences vs DataStore

Таблица сравнения

Решение	Для чего
Room	большие данные
DataStore	настройки
SharedPreferences	устарело

SharedPreferences проблема

- ✗ блокирует поток
 - ✗ нет реактивности
 - ✗ легко ломается
-

16. DataStore (современная замена prefs)

Используется для:

- токенов
 - настроек
 - флагов
-

17. Частые ошибки новичков

✗1. блокируют UI

→ делают запросы в main thread

✗2. не используют Flow

→ данные не обновляются автоматически

✗3. смешивают API и DB модели

→ архитектурный хаос

✗4. нет миграций

→ crash после обновления приложения

18. Реальный кейс

Проблема

- приложение показывает пустой список после рестарта
-

Причина

- данные не сохранялись в Room
 - использовался только API
-

Решение

□ API → save to Room → UI reads from Room

□

Итог

- данные сохраняются

- оффлайн работает
 - UI быстрее
-

19. Как думает senior разработчик

Перед выбором хранения он спрашивает:

- нужны ли данные оффлайн?
 - это кеш или источник истины?
 - нужны ли реактивные обновления?
 - как будет работать миграция?
-

20. Итог лекции

Главное понимание

- ☐ Room = локальная база данных, которая делает SQLite безопасной, типобезопасной и реактивной
 - ☐
-

Что нужно запомнить

- Room = Entity + DAO + Database
 - Flow = реактивные данные
 - offline-first = стандарт
 - миграции обязательны
 - UI не должен напрямую ходить в API
-

Если упростить

Room - это способ заставить приложение помнить данные, даже если сервер или интернет исчезли

□ Лекция 7. Dependency Injection (Hilt / Koin)

□ Цель лекции

Понять, почему “создавать всё через `new` внутри классов” - путь к хаосу, и как Dependency Injection делает архитектуру Android-приложения масштабируемой, тестируемой и управляемой.

□ Главная идея

В нормальной архитектуре класс **не должен сам создавать свои зависимости**.

Он должен их получать извне.

- Объект не “строит мир вокруг себя”, а “получает готовые инструменты”

1. Проблема без DI

✗ Плохой пример

```
class UserRepository {  
  
    private val api = Retrofit.Builder()  
        .baseUrl("https://api.example.com")  
        .build()  
        .create(ApiService::class.java)  
  
}
```

□

□ Что здесь не так

- жесткая связка с Retrofit
 - невозможно подменить API в тестах
 - нельзя переиспользовать
 - нарушение SRP (Single Responsibility Principle)
-

Итог

класс становится “монстром”, который знает слишком много

2. Что такое Dependency Injection

□ Простое определение

Dependency Injection (DI) - это:

способ передавать зависимости в класс извне, а не создавать их внутри

✓ Правильный вариант

```
□ class UserRepository(  
    private val api: ApiService  
)  
□
```

Что изменилось

- класс стал “чистым”
 - зависимости управляются снаружи
 - код стал тестируемым
-

3. Поток зависимостей

□ App → создает зависимости → передает в Repository → ViewModel → UI

□

4. Зачем DI в Android

Реальные проблемы без DI

- невозможно подменить API
 - сложно писать тесты
 - дублирование Retrofit/DB
 - хаос в создании объектов
-

DI решает:

- ✓ централизованное создание объектов
 - ✓ переиспользование
 - ✓ тестируемость
 - ✓ масштабируемость
-

5. Основные типы DI

Тип	Описание
Constructor Injection	через конструктор
Field Injection	в поле класса
Method Injection	через методы

✓ В Android стандарт - Constructor Injection

6. DI вручную (без библиотек)

Пример

```
❑ val api = RetrofitClient.api  
  
val repository = UserRepository(api)  
  
val viewModel = UserViewModel(repository)  
  
❑
```

❑ Проблема

- много boilerplate
 - сложно управлять зависимостями
 - нет масштабируемости
-

7. Dependency Injection фреймворки

В Android есть 2 основных решения:

❑ Hilt (Google)

□ Koin (Kotlin-first)

8. Hilt - стандарт индустрии

□ Что это

Hilt - это:

официальная DI-библиотека от Google, построенная поверх Dagger

Почему он важен

- compile-time DI (ошибки ловятся при сборке)
 - высокая производительность
 - интеграция с ViewModel / Activity
-

9. Как работает Hilt (простая схема)

□ Module → предоставляет зависимости → Hilt Graph → Injection в классы

□

10. @Inject - базовый механизм

```
class UserRepository @Inject constructor(  
    private val api: ApiService  
)  
  

```

Что происходит

Hilt сам понимает:

“этот класс можно создавать автоматически”

11. Modules - источник зависимостей

```
@Module  
  
@InstallIn(SingletonComponent::class)  
  
object NetworkModule {  
  
    @Provides  
  
    fun provideApiService(): ApiService {  
        return RetrofitClient.api  
    }  
}  
  

```

Что это значит

- ты объясняешь Hilt'у, как создавать объект
 - Hilt хранит его в графе зависимостей
-

12. @HiltViewModel

□@HiltViewModel

```
class UserViewModel @Inject constructor(  
    private val repository: UserRepository  
) : ViewModel()  
  
□
```

Важно

ViewModel больше не создаётся вручную.

13. Жизненный цикл зависимостей

Scope	Живёт пока
-------	------------

Singleton	всё приложение
-----------	----------------

Activity экран

ViewModel ViewModel

❑ Ошибка новичков

✗ используют Singleton везде

→ утечки памяти

14. Koin - альтернатива Hilt

❑ Что это

Koin - это:

DI-фреймворк, написанный на Kotlin, без генерации кода

Главное отличие

Hilt	Koin
------	------

compile-time	runtime
--------------	---------

сложнее	проще
---------	-------

быстрее медленнее

15. Koin module

```
val appModule = module {  
  
    single { RetrofitClient.api }  
  
    single { UserRepository(get()) }  
  
    viewModel { UserViewModel(get()) }  
}  
  

```

Что происходит

- `get()` = взять зависимость из контейнера
 - всё резолвится во время выполнения
-

16. Как внедряется Koin

```
□startKoin {  
    modules(appModule)  
}  
□
```

17. Hilt vs Koin - сравнение

Критерий	Hilt	Koin
Производительность	высокая	средняя
Простота	сложнее	проще
Ошибки	compile-time	runtime
Поддержка Google	да	нет

18. Когда что использовать

✓Hilt

- большие проекты
 - production apps
 - команды
 - долгосрочная архитектура
-

✓Koin

- пет-проекты
 - MVP/стартап
 - быстрые прототипы
-

19. Частые ошибки новичков

✗1. создают зависимости внутри классов

→ ломают DI

✗2. делают всё Singleton

→ утечки памяти

✗3. смешивают Hilt и manual DI

→ архитектурный хаос

X4. не понимают score

→ странное поведение объектов

20. Реальный кейс

Проблема

- ViewModel создаёт Repository сам
 - невозможно подменить API в тестах
-

Решение

□ `Hilt -> inject ApiService -> inject Repository -> inject ViewModel`

□

Итог

- тестируемость выросла
 - код стал чище
 - архитектура масштабируется
-

21. Как думает senior разработчик

Перед созданием объекта он спрашивает:

- кто должен владеть этой зависимостью?
 - можно ли её переиспользовать?
 - как я буду это тестировать?
 - где должен жить этот объект?
-

22. Итог лекции

Главное понимание

□ Dependency Injection = способ строить приложение так, чтобы классы не создавали зависимости сами, а получали их извне через централизованный контейнер

□

Что нужно запомнить

- DI убирает `new` из бизнес-логики
 - Hilt = production стандарт
 - Koin = простой и быстрый старт
 - constructor injection = лучший подход
 - scope = критически важно
-

Если упростить

DI - это способ перестать “собирать приложение руками” в каждом классе

□ Лекция 8. Архитектура Android: MVVM, MVI и Clean Architecture

□ Цель лекции

Понять, почему “просто написать код в Activity” перестаёт работать на реальных проектах, и как архитектурные подходы помогают удерживать приложение управляемым, тестируемым и расширяемым.

□ Главная идея

Когда проект маленький - архитектура кажется лишней.

Когда проект растёт - отсутствие архитектуры превращает код в хаос.

- Архитектура - это не про “красоту кода”, а про контроль сложности
-

1. Почему без архитектуры всё ломается

✗ Типичный хаос новичка

□ Activity:

→ сеть

→ UI

→ логика

→ база данных

→ обработка ошибок

□

□ Что происходит через 2–3 месяца

- код невозможно читать
 - баги появляются “везде”
 - любое изменение ломает другое
 - тестирование почти невозможно
-

Главная проблема

всё связано со всем

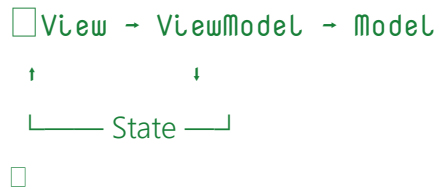
2. MVVM - базовый стандарт Android

□ Что такое MVVM

MVVM =

- Model - данные
 - View - UI
 - ViewModel - логика экрана
-

Схема



3. Роли компонентов MVVM

□ View (Activity / Fragment)

- отображает UI
- слушает состояние
- не содержит бизнес-логики

□ ViewModel

- хранит состояние
- обрабатывает логику экрана
- работает с репозиториями

□ Model

- данные
- API
- база данных

4. Пример MVVM

```
class UserViewModel(  
    private val repository: UserRepository  
) : ViewModel() {  
  
    val users = MutableStateFlow<List<User>>(emptyList())  
  
    fun loadUsers() {  
        viewModelScope.launch {  
            users.value = repository.getUsers()  
        }  
    }  
}
```

5. Почему MVVM работает

✓преимущества

- UI отделён от логики
 - проще тестировать
 - проще поддерживать
 - стандарт Android Jetpack
-

✗проблема MVVM

со временем ViewModel начинает разрастаться

6. Clean Architecture - следующий уровень

□ Идея

Clean Architecture разделяет приложение на слои:

каждый слой знает только о внутреннем, но не о внешнем

7. Слои Clean Architecture

□ UI Layer → Domain Layer → Data Layer

□

□ UI Layer

- Activity / Fragment
 - ViewModel
 - UI state
-

□ Domain Layer (сердце логики)

- UseCases
 - бизнес-правила
 - не зависит от Android
-

□ Data Layer

- Retrofit API
 - Room DB
 - Repository implementation
-

8. Почему Clean Architecture важна

✓ преимущества

- легко тестировать
 - легко менять API или DB
 - масштабируется на большие команды
 - бизнес-логика не зависит от Android
-

□ ключевая идея

UI не должен знать, откуда пришли данные

9. Пример Clean Architecture

UseCase (Domain слой)

```
□ class GetUsersUseCase(  
    private val repository: UserRepository  
) {  
    suspend operator fun invoke(): List<User> {  
        return repository.getUsers()  
    }  
}
```

□

ViewModel

```
□ class UserViewModel(  
    private val getUsers: GetUsersUseCase
```

```
) : ViewModel() {  
  
    val state = MutableStateFlow<List<User>>(emptyList())  
  
    fun load() {  
        viewModelScope.launch {  
            state.value = getUsers()  
        }  
    }  
}
```

10. MVI - реактивная архитектура

□ Что такое MVI

MVI =

- Model
 - View
 - Intent
-

Главное отличие

однонаправленный поток данных

11. Поток MVI

□ View → Intent → ViewModel → State → View

□

12. Пример MVI

Intent (действия пользователя)

```
sealed class UserIntent {  
    object LoadUsers : UserIntent()  
}
```

State (состояние UI)

```
data class UserState(  
    val isLoading: Boolean = false,  
    val users: List<User> = emptyList()  
)
```

ViewModel

```
class UserViewModel : ViewModel() {  
  
    private val _state = MutableStateFlow(UserState())  
    val state = _state.asStateFlow()  
  
    fun handleIntent(intent: UserIntent) {  
        when (intent) {  
            is UserIntent.LoadUsers -> loadUsers()  
        }  
    }  
  
    private fun loadUsers() {  
        viewModelScope.launch {  
            _state.value = UserState(isLoading = true)  
  
            val users = loadFromApi()  
        }  
    }  
}
```

```
        _state.value = UserState(  
            isLoading = false,  
            users = users  
        )  
    }  
}  
}
```

13. MVVM vs MVI

Критерий	MVVM	MVI
поток	двунаправленны й	однонаправленны й
сложность	ниже	выше
контроль состояния	средний	строгий
дебаг	сложнее	проще
масштаб	средний	большой

14. Когда что использовать

✓MVVM

- большинство приложений
- стандарт Android
- быстрый старт

✓Clean + MVVM

- production apps
- большие команды
- сложная бизнес-логика

✓MVI

- сложные UI
- финтех
- чаты
- realtime системы

15. Частые ошибки новичков

✗1. бизнес-логика в Activity

→ невозможно поддерживать

✗2. огромный ViewModel

→ "God Object"

✗3. отсутствие слоёв

→ смешаны API, UI, DB

X4. нет единого состояния UI

→ баги синхронизации

16. Реальный кейс из практики

Проблема

- экран пользователя содержит:
 - API запросы
 - фильтрацию
 - кеширование
 - UI логику
-

Что произошло

- ViewModel разросся до 1200 строк
 - баги невозможно локализовать
-

Решение

- вынесли UseCases
 - разделили Data Layer
 - ввели StateFlow
 - убрали бизнес-логику из ViewModel
-

Итог

- код стал читаемым
 - тесты появились
 - баги локализуются быстро
-

17. Как думает senior разработчик

Перед написанием кода он задаёт вопросы:

- где должна жить логика?
 - можно ли это протестировать отдельно?
 - какой слой за это отвечает?
 - не смешиваю ли я ответственность?
-

18. Итог лекции

Главное понимание

☐ Архитектура = способ разделить приложение на слои, чтобы каждый отвечал только за свою часть и не ломал остальные

☐

Что нужно запомнить

- MVVM = стандарт Android
 - Clean Architecture = масштабируемость
 - MVI = строгий контроль состояния
 - ViewModel не должен быть “богом”
 - UI не должен знать источник данных
-

Если упростить

архитектура - это способ не дать приложению превратиться в хаос через 3 месяца разработки

□ Лекция 9. Публикация приложения в Google Play и релизный билд

□ Цель лекции

Понять полный путь приложения от локальной сборки до публикации в Google Play, научиться правильно готовить релиз и разбираться, почему “у меня на дебаге работает, а у пользователей - нет”.

□ Общая картина

Публикация - это не “нажать кнопку Upload”.

Это цепочка этапов:

□ Код → Debug Build → Release Build → Подпись → AAB → Google Play → Модерация →

Пользователи

□

1. Debug vs Release - ключевое различие

□ Debug сборка

Используется для разработки.

Особенности:

- логирование включено
 - отладка доступна
 - нет оптимизаций
 - медленнее
 - можно подключать debugger
-

□ Release сборка

То, что получают пользователи.

Особенности:

- код оптимизирован (R8/ProGuard)
 - логи могут быть отключены
 - код обфусцирован
 - быстрее
 - нельзя отлаживать как debug
-

□ Классическая ошибка новичков

“У меня в debug всё работает” ≠ “в release всё будет работать”

2. Подпись приложения (Signing)

□ Зачем нужна подпись

Android должен быть уверен:

- кто автор приложения

- что APK/AAB не изменяли
-

□ Keystore

Это файл с приватным ключом:

- .jks
 - .keystore
-

□ Критически важно

Если потерял keystore:

- ✗ нельзя обновить приложение в Google Play
 - ✓ только выпуск нового приложения
-

Пример генерации

```
□ keytool -genkey -v -keystore release.jks -keyalg RSA -keysize 2048 -  
validity 10000 -alias my_alias  
□
```

3. Gradle конфигурация релиза

```
□ android {  
  
    signingConfigs {  
        create("release") {  
            storeFile = file("release.jks")  
            storePassword = System.getenv("STORE_PASS")  
            keyAlias = System.getenv("ALIAS")  
        }  
    }  
}
```

```
        keyPassword = System.getenv("KEY_PASS")
    }
}

buildTypes {
    release {
        isMinifyEnabled = true           // включает R8
        isShrinkResources = true         // удаляет мусорные ресурсы

        proguardFiles(
            getDefaultProguardFile("proguard-android-
optimize.txt"),
            "proguard-rules.pro"
        )

        signingConfig = signingConfigs.getByName("release")
    }
}
}
```

□

4. Что делает R8 / ProGuard

□ Это оптимизатор кода

Он:

- удаляет неиспользуемый код
 - сокращает имена классов
 - уменьшает размер приложения
 - усложняет reverse engineering
-

□ возможные проблемы

- ломается рефлексия
 - падает JSON парсинг
 - не работают аннотации
 - краши только в release
-

✓решение

ProGuard rules:

```
□-keep class com.example.model.* { *; }  
□
```

5. APK vs AAB

✗APK (устаревающий формат)

- один файл
 - содержит всё сразу
 - тяжелее
 - не оптимизирован под устройство
-

✓AAB (Android App Bundle)

Google Play стандарт.

Что делает Google:

- собирает APK под конкретное устройство
 - удаляет лишние ресурсы
 - уменьшает размер
-

☐ ИТОГ

AAB = умная доставка приложения пользователю

6. Подготовка перед публикацией

✓чеклист разработчика

- нет crash при запуске
 - нет утечек памяти
 - отключены debug логи
 - протестированы разные устройства
 - проверены оффлайн сценарии
 - проверены слабые сети
-

7. Google Play Console

☐ шаги публикации:

1. Создание приложения

- название
 - язык
 - тип (app/game)
-

2. Store Listing

- описание
- иконка (512x512)
- скриншоты

- feature graphic
-

3. Content rating

- возрастные ограничения
 - анкета
-

4. Privacy Policy

обязательна почти всегда

5. Upload AAB

6. Release track

- internal testing
 - closed testing
 - open testing
 - production
-

8. Процесс модерации

☐ Google проверяет:

- стабильность
 - разрешения
 - политику данных
 - UX
 - безопасность
-

□ сроки:

- от нескольких часов
 - до 7 дней
-

✗ частые причины отказа

- краш при запуске
 - нет privacy policy
 - агрессивные permissions
 - сбор данных без объяснения
 - неработающие функции
-

9. Что происходит после релиза

□ доставка приложения

- пользователи скачивают AAB-версии
 - Google генерирует APK под устройство
 - обновления приходят через Play Store
-

□ важные метрики

- crash rate
 - ANR rate
 - retention
 - ANR (Application Not Responding)
-

10. Реальный кейс из практики

Проблема

- в debug всё работает
 - в release приложение падает при запуске
-

Причина

R8 удалил класс, используемый через reflection.

Решение

```
□ -keep class com.example.analytics.** { *; }  
□
```

Итог

- релиз восстановлен
 - добавлены ProGuard правила
 - тестирование релиза стало обязательным
-

11. Частые ошибки новичков

✗1. нет теста release-сборки

→ баги уходят в прод

✗2. потерян keystore

→ невозможно обновить приложение

✗3. игнор ProGuard

→ краши только у пользователей

✗4. нет проверки на слабых устройствах

→ приложение "тормозит" у части аудитории

12. Как думает senior

Перед релизом он проверяет:

- что может сломаться в проде?
 - где используется reflection?
 - что удалит R8?
 - как ведёт себя приложение без сети?
 - что увидит пользователь при первом запуске?
-

13. Итог лекции

Главное понимание:

☐ Релиз - это не финальный этап разработки, а отдельная инженерная зона риска

☐

Запомнить:

- debug ≠ release
- keystore = критически важен
- AAB = стандарт Google Play
- R8 может ломать код
- Google проверяет не только код, но и продукт

□ Лекция 10. Жизненный цикл фичи в Android-проекте (от задачи до продакшена)

□ Цель лекции

Понять, как реально “живет” фича в продуктовой разработке: от идеи в таск-трекере до релиза, и почему код - это только середина процесса.

□ Общая картина

Фича в Android - это не “написал экран и готово”.

Это цепочка взаимодействий:

□ Требование → Декомпозиция → Оценка → Разработка → Code Review → QA → Багфиксы
→ Релиз → Мониторинг

□

1. Откуда берётся задача

□ Источники задач:

- продукт (Product Owner)
 - бизнес
 - аналитика
 - поддержка (support)
 - баги от пользователей
-

□ проблема новичков

задача воспринимается как “написать экран”

На деле это:

- логика
 - edge cases
 - интеграции
 - ошибки
 - аналитика
 - поведение в проде
-

2. Анализ требований (самый важный этап)

! Что делает разработчик/QA до кода

- уточняет сценарии
- ищет неоднозначности
- проверяет ограничения
- выясняет edge cases

□ правильные вопросы

- что если нет интернета?
- что если API упало?
- что если пустой список?
- можно ли повторить действие?
- какие ограничения по данным?

□ типичная ошибка

“в задаче не было написано - значит не делаем”

Senior подход:

“если пользователь может это сломать - он это сломает”

3. Декомпозиция задачи

□ зачем нужна декомпозиция

Большая задача = плохая оценка + высокий риск ошибок

□ пример фичи: “Экран списка пользователей”

Разбивка:

- UI экран
- + API слой
- + модель данных
- + ViewModel
- + состояние (Loading/Error/Empty)

- + обработка ошибок
- + кеширование
- + тестирование



□ пример оценки

задача	время
UI	2ч
API	2ч
ViewModel	3ч
состояния	2ч
тестирование	2ч

□ ИТОГ

оценка становится реальной, а не “на глаз”

4. Разработка фичи

□ правильный порядок мышления

1. данные (API / DB)
2. бизнес-логика (ViewModel / UseCase)
3. состояние UI
4. отображение

✗ частая ошибка

начать с UI без понимания данных

5. Code Review

□ что это на самом деле

Code Review - это:

проверка качества, архитектуры и потенциальных багов до попадания в прод

✓ что проверяют

- читаемость кода
 - архитектуру
 - дублирование
 - edge cases
 - безопасность
 - производительность
-

✗ что это НЕ

- “вкус кода”
 - личная критика
 - формальность
-

□ хороший PR содержит:

- описание фичи
 - скриншоты
 - сценарии тестирования
 - список изменений
-

6. QA цикл (очень важный этап)

□ что делает QA

QA не “нажимает кнопки”.

QA:

- проверяет требования
 - ищет несоответствия
 - ломает сценарии
 - проверяет негативные кейсы
 - проверяет интеграции
-

□ уровни тестирования

1. Smoke

- приложение запускается?

2. Functional

- работает ли фича?

3. Regression

- не сломали ли старое?

4. Edge cases

- нестандартные сценарии
-

☐ пример edge case

- потеря сети во время запроса
 - поворот экрана
 - убийство процесса системой
 - пустой ответ API
 - медленный сервер
-

7. Жизнь багов

☐ цикл бага:

☐ Найден → Заведен → В работе → Исправлен → Проверен → Закрыт

☐

☐ уровни критичности

- Blocker - всё сломано
 - Critical - фича не работает
 - Major - важная проблема
 - Minor - косметика
-

☐ типичная проблема

баг “не воспроизводится у разработчика”

☒ решение

- логи
 - шаги воспроизведения
 - окружение
 - видео
-

8. Релиз фичи

☐ что происходит перед релизом

- QA sign-off
 - проверка багов
 - regression test
 - проверка релизной сборки
 - согласование с продуктом
-

☐ правило индустрии

“если есть критические баги - релиз не идёт”

(но в реальности бывают компромиссы)

9. Мониторинг после релиза

☐ что смотрят:

- crash rate
- ANR
- ошибки API
- поведение пользователей
- аналитика событий

□ важная идея

релиз - это не конец, а начало наблюдения

10. Реальный кейс

проблема

Фича “оплата”

- в тесте работает
 - в проде падает у 5% пользователей
-

причина

- медленный интернет
 - таймаут API
 - нет обработки retry
-

ВЫВОД

QA не протестировал:

- слабую сеть
 - таймауты
 - повтор запроса
-

11. Типичные ошибки команд

X1. нет декомпозиции

→ оценка неверная

X2. нет QA мышления у разработчиков

→ баги доходят до прод

X3. игнор edge cases

→ "редкие" баги становятся массовыми

X4. отсутствие логов

→ невозможно понять проблему

12. Как думает senior разработчик

Перед началом он спрашивает:

- что может пойти не так?
- как это сломают пользователи?
- как это будет вести себя в плохой сети?
- что увидит QA?
- что будет в проде через месяц?

13. Итог лекции

Главное понимание:

□ Фича - это не код. Это процесс доставки ценности от идеи до стабильной работы в продакшене

□

Запомнить:

- задача начинается с анализа, а не кода
 - декомпозиция снижает риски
 - QA - часть инженерного процесса, а не “последний этап”
 - баги - это нормальная часть системы
 - релиз не завершает работу
-

Финальный вывод серии

хороший Android-разработчик не просто пишет код - он управляет жизненным циклом фичи от идеи до стабильного поведения в проде

Курсовой проект

Создать приложение которое выводит на экран результат будущих матчей премьер лиги Англии.

Дизайн можно выбирать свободно, будет плюсом использование принципов Material UI.

Технические детали:

- Для получения будущих матчей использовать следующий API (GET) <https://fixturedownload.com/feed/json/epl-2023>.
- Использовать библиотеку Retrofit или аналог для получения данных с сервера.
- Использовать корутины ([Kotlin Coroutines](#)) для работ со сетью.
- Использовать предпочитаемую архитектуру из MVP, MVVM, MVI.
- Использовать LiveData/Flow.
- Использовать Single Activity подход.

Детали для экранов:

- API возвращает дату и время в значении UTC под именем "DateUtc". Его необходимо конвертировать в локальное время устройства.
- На экране списка матчей необходимо отобразить список матчей с информацией о командах, счете и даты проведения матча.
- На экране деталей необходимо отобразить как минимум название команд, счет, дату, время и локацию.

Будет большим плюсом:

- Написание юнит тестов.
- Использование Jetpack Compose вместо XML.
- Добавление кеширования.
- Добавить пагинацию.
- Добавления локального поиска матча по имени команды.

Итог

- Приложение состоит из 2 экранов:
 1. Экран список матчей
 2. Экран детали матча
- При нажатии на элемент списка переходит на экран деталей матча.

Что почитать?

[Язык программирования Kotlin](#)

[app\assets\AppAsset::register\(\\$this\); ?> Здесь мы собираем ресурсы по Котлину и переводим документацию. Сообщество открыто для новых участников - любого, кто может переводить и проверять перевод. Редактирование текста происходит похожим на википедию образом, с той лишь разницей, что тексты и структура меню хранятся в Git. Внимание.](#)

<https://kotlinlang.ru/>

[Язык программирования Kotlin](#) - НЕ официальная русскоязычная страница Kotlin с руководством по языку.

[Kotlin Programming Language](#)

[Easy to pick up, so you can create powerful applications immediately. Concise data class Employee\(val name: String, val email: String, val company: String \) // + automatically generated equals\(\), hashCode\(\), toString\(\), and copy\(\) object MyCompany { // A singleton const val name: String = "MyCompany" } fun main\(\) { // Function at the top level val employee = Employee\("Alice", // No `new` keyword "alice@mycompany.com", MyCompany.name\) println\(employee\) } Safe fun reply\(condition: Boolean\): String?](#)

<https://kotlinlang.org/>

[Kotlin](#) - официальная страница языка Kotlin на английском языке с ссылкой на [документацию](#), [инструкцию по установке](#) и [онлайн компилятор](#).

[Documentation | Android Developers](#)

[This set of libraries provides APIs for essential app architecture tasks like lifecycle management and data persistence, so you can write modular apps with less boilerplate code. The Android Support Library offers backward-compatible versions of a number of features, including others not built into the framework.](#)

<https://developer.android.com/docs>

[Documentation | Android Developers](#) - тут можно найти всё что касается разработки под устройства работающими на операционной системе Android, Wear OS, Android TV или Android for Cars. Документации, инструменты, гайдлайны и best practices с примерами кода.

[Android уроки | Бесплатные курсы для чайников | Android приложение с нуля](#)

[9 из 10 человек пользуются смартфонами под управлением операционной системы Android! Неудивительно, что популярность разработки для этой ОС](#)

[постоянно растёт. Думаете над тем, чтобы начать путь мобильного разработчика? Предлагаем вашему вниманию курс "Разработка под Android для начинающих", созданный специалистами Google \(владельцами Android и всего хорошего в мире ИТ\) для платформы Udacity.](#)

https://javarush.ru/quests/QUEST_GOOGLE_ANDROID

[Программирование под Андроид на JavaRush](#) - перевод известной образовательной программы Google, опубликованной на иностранной платформе Udacity. С ее помощью слушатели смогут изучить ООП и научиться создавать современные интерактивные приложения.

[Основы Kotlin. Введение](#)

[Языки программирования - интереснейшая область современной техники. За последние 30-40 лет информационные технологии разрослись до невероятных пределов, и сейчас мало кто в состоянии обозреть эту область в полном объёме.](#)

<https://www.fandroid.info/osnovy-kotlin-vvedenie/>

[Fandroid](#) - русскоязычный ресурс для изучения основ Kotlin.

Основной принцип успешного обучения это понимание того, с чем вы работаете. Нужно всегда разбирать код который вы ~~скопировали~~ написали, каждую строку, каждый используемый компонент. Да-да первым делом для ускорения обучения вы будете копировать куски кода, но это только затянет его в будущем, поэтому даже если вы видите максимально простой пример кода в уроках, пишите его самостоятельно. Так же одна из рекомендаций - задавайте себе больше вопросов по коду, это позволит вам не упустить важные моменты при старте и построить надёжный фундамент для более сложных уровней разработки в будущем.

Интерактивная игра для знакомства с GIT

Используй смекалку

[Learn Git Branching](#)

[An interactive Git visualization tool to educate and challenge!](#)

https://learngitbranching.js.org/?locale=ru_RU

Инструменты

☐ Android Studio

[Download Android Studio & App Tools - Android Developers](#)

Не обязательно устанавливать последнюю версию. Для изучения подойдут более старые дистрибутивы которые можно [скачать отдельно](#) (сначала необходимо принять соглашение в конце страницы).

Стайлгайд

[Kotlin style guide | Android Developers](#)

Базовый гайд от Google по оформлению кода на Kotlin.

Гайд по оформлению кода от создателей языка Kotlin.

[GitHub - RedMadRobot/kotlin-style-guide: red_mad_robot Kotlin Style Guide](#)

Каждая команда адаптирует под себя или расширяет уже существующие принципы написания кода. Вот один из примеров такой адаптации.

Именно в таком виде мы ожидаем увидеть код в ваших проектах.